

13장 포인터와 배열

1. 포인터와 배열의 관계
2. 포인터 연산
3. 상수형태의 문자열을 가리키는 포인터
4. 포인터 배열



배열명의 의미

3

- 배열명 : 배열 첫번째 요소의 **주소**(변수가 아니고 **상수**임)
- 배열명이 가리키는 자료형 : 첫번째 배열 요소의 자료형
- 배열명의 자료형 : (첫번째요소자료형)*

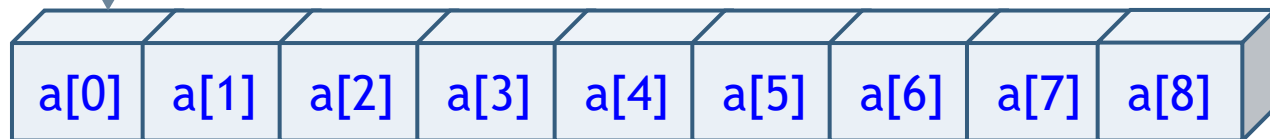
`int a[9];`

`a`

배열명 `a` -> `&a[0]`

배열명 `a`가 가리키는 자료형 -> `int`

배열명 `a`의 자료형 -> `int*`



100 104 108 112 116 ...



차이 4 -> 배열은 모든 요소가 연속적으로 할당



배열명의 의미

4

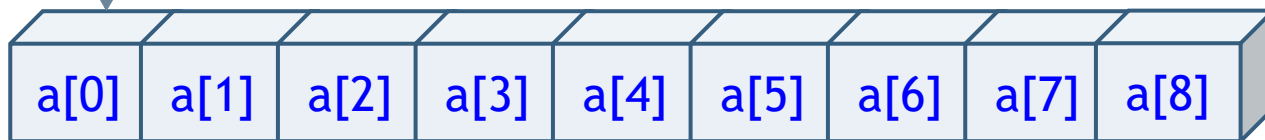
double a[9];

a

배열명 a -> **&a[0]**

배열명 a가 가리키는 자료형 -> ()

배열명 a의 자료형 -> ()



100 () () () () ...



차이 () -> 배열은 모든 요소가 연속적으로 할당



배열명의 의미

5

- 12장에서 배운 포인터 -> 변수 -> **포인터 변수**
- 배열명 -> 주소상수 -> **포인터 상수**
- 상수이던지 변수이던지 주소를 나타내면 **모두 포인터라 부름**



예제 1 : 배열명의 의미

6

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int a[5] = { 10, 20, 30, 40, 50 };
```

```
    printf("배열명 = %p\n", a);
```

```
    printf("첫번째 원소의 주소 = %p\n", &a[0]);
```

```
    printf("*a = %d\n", *a);
```

```
    printf("a[0] = %d\n", a[0]);
```

```
    a = 1300000;
```

```
    return 0;
```

```
}
```

배열명 = 11533644

첫번째 원소의 주소 = 11533644

*a = 10

a[0] = 10

// 에러: 배열명 a는 상수이므로 대입 불가



예제 2: 배열의 할당

7

```
#include <stdio.h>
int main(void)
{
    int arr[3] = { 0, 1, 2 };
    printf("배열의 이름: %p \n", arr);
    printf("첫 번째 요소: %p \n", &arr[0]);
    printf("두 번째 요소: %p \n", &arr[1]);
    printf("세 번째 요소: %p \n", &arr[2]);
    return 0;
}
```

배열의 이름: 00B2FCD0
첫 번째 요소: 00B2FCD0
두 번째 요소: 00B2FCD4
세 번째 요소: 00B2FCD8

*각 요소의 주소값의 차이가 4임
-> 배열의 요소는 메모리에
연속적으로 할당되어 있음을
의미*



배열명과 간접참조연산*

8

- 배열명은 배열의 첫번째 요소의 주소이므로 간접참조연산 * 가능함
- *배열명 => 첫번째요소의값=> 배열명[0]

```
int a[10]; -> *a <=> a[0]
```




예제 3: *배열명

9

```
#include <stdio.h>
int main(void)
{
    int arr1[3] = { 1, 2, 3 };
    double arr2[3] = { 1.1, 2.2, 3.3 };
    printf("%d %g \n", *arr1, *arr2);
    printf("%d %g \n", arr1[0], arr2[0]);
    *arr1 += 100;
    *arr2 += 120.5;
    printf("%d %g \n", *arr1, *arr2);
    printf("%d %g \n", arr1[0], arr2[0]);
    return 0;
}
```

1	1.1
1	1.1
101	121.6
101	121.6

arr1이 가리키는 자료형 : int
arr2이 가리키는 자료형 : double
*arr1의 자료형 : int**
*arr2의 자료형 : double**
**arr1 : arr1주소에서 4바이트공간*
**arr2 ; arr2주소에서 8바이트공간*
%g : 너무 크거나 작은 수는 %e,
그 외는 %f



포인터와 배열

10

- 포인터변수를 배열명처럼 사용가능 -> 즉, 포인터변수를 배열명처럼 인덱스와 함께 사용 가능함 -> **포인터[인덱스]**
- 배열명은 상수이고 포인터는 변수라는 차이외에는 유사함 -> Why?

```
int a[10];  
int* ptr = a;  
-> ptr[0] <=> a[0]  
-> ptr[1] <=> a[1]  
-> ptr[2] <=> a[2]  
...  
-> ptr[n] <=> a[n]
```



예제 4 : 포인터를 배열처럼 사용

11

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int arr[3] = { 15, 25, 35 };
```

```
    int* ptr = &arr[0];    // int* ptr=arr; 과 동일한 문장
```

```
    printf("%d %d \n", ptr[0], arr[0]);
```

```
    printf("%d %d \n", ptr[1], arr[1]);
```

```
    printf("%d %d \n", ptr[2], arr[2]);
```

```
    printf("%d %d \n", *ptr, *arr);
```

```
    return 0;
```

```
}
```

15	15
25	25
35	35
15	15

ptr은 포인터이지만 배열처럼 사용가능



포인터 연산

12

- 가능한 연산: **간접참조(*), 정수덧셈(+), 정수뺄셈(-), 증감(++ , --)**
- 피연산자가 포인터(주소)일때 산술 연산 +, - 와 의미가 다름

수식(p :포인터, n:정수)	의미
p + n	p + n*(포인터가 가리키는 자료형의 크기)
p - n	p - n*(포인터가 가리키는 자료형의 크기)
p ++	p = p + (포인터가 가리키는 자료형의 크기)
p --	p = p - (포인터가 가리키는 자료형의 크기)

포인터가 가리키는 자료형	자료형 크기
char	1
int	4
double	8



포인터 연산

13

```
int* p;  
p = (int*)1000;           // p=>1000  
p = p + 1;                // p=>1000+1*4=>1004  
p = p - 2;                // p=>1004-2*4=>996  
p++;                      // p=>996+1*4=>1000  
p--;                      // p=>1000-1*4=>996
```

주의사항 : 2번째 줄의 코드를 아래처럼 쓰면 오류 발생

p = 1000; // 포인터(int) = 정수(int) -> 자료형 불일치
 // 피연산자가 포인터인 경우 자동 형변환 안됨*



포인터 연산

14

```
double* p;  
p = (double*)1000;    // p=>?  
p = p + 1;            // p=>?  
p = p - 2;            // p=>?  
p++;                  // p=>?  
p--;                  // p=>?
```

주의사항 : 2번째 줄의 코드를 아래처럼 쓰면 에러 발생

p = 1000; // 포인터(double) = 정수(int) -> 자료형 불일치
 // 피연산자가 포인터인 경우 자동 형변환 안됨*



예제 5: 포인터 연산

15

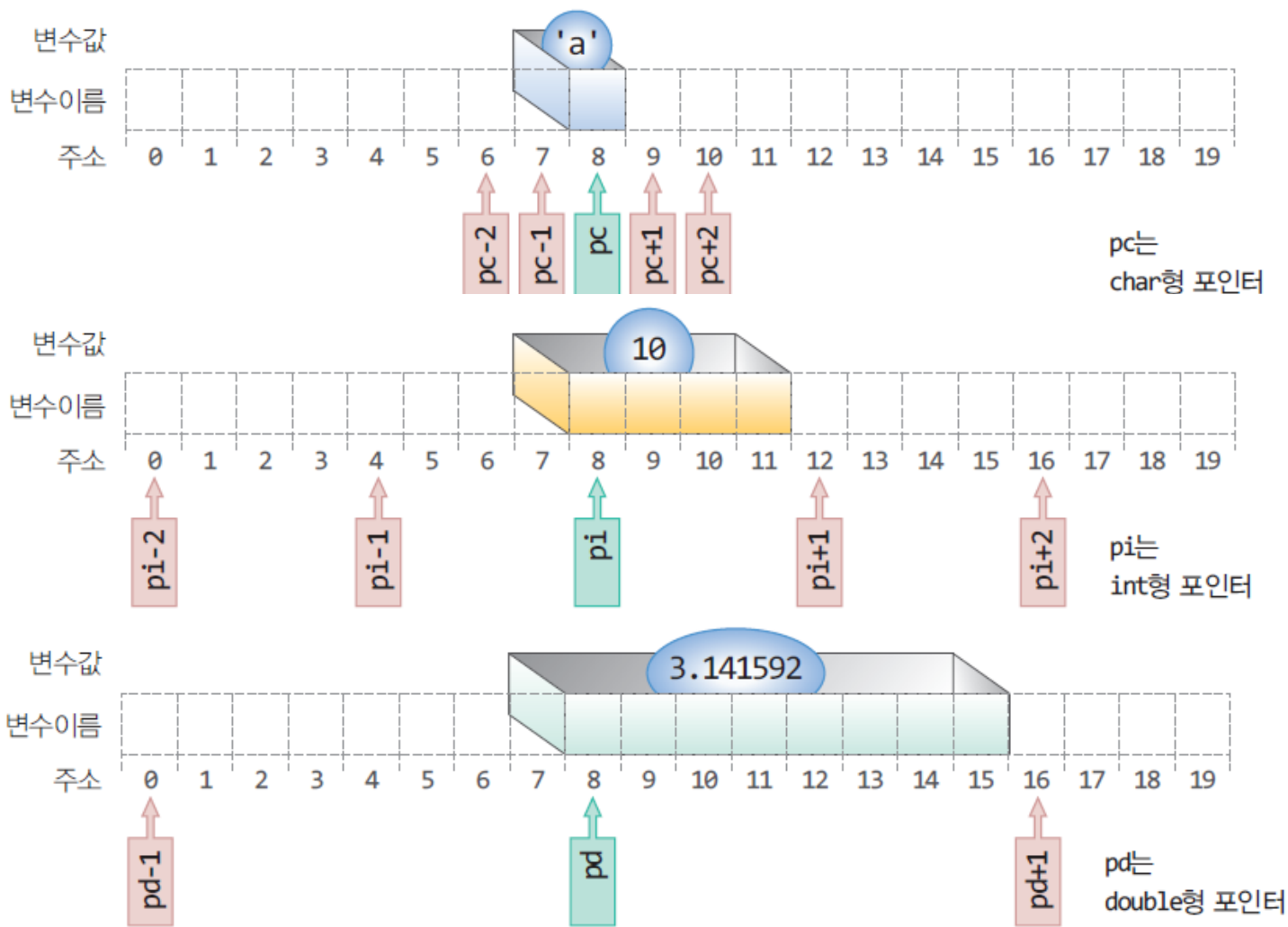
```
#include <stdio.h>
int main(void)
{
    char* pc;
    int* pi;
    double* pd;
    pc = (char*)10000;
    pi = (int*)10000;
    pd = (double*)10000;
    printf("증가 전 pc = %u, pi = %u, pd = %u\n", pc, pi, pd);
    pc++;
    pi++;
    pd++;
    printf("증가 후 pc = %u, pi = %u, pd = %u\n", pc, pi, pd);
    printf("pc+2=%u, pi+2=%u, pd+2=%u\n", pc+2, pi+2, pd+2);
    return 0;
}
```

증가 전 pc = 10000, pi = 10000, pd = 10000
증가 후 pc = 10001, pi = 10004, pd = 10008
pc+2= 10003, pi+2= 10012, pd+2= 10024



포인터 연산

16





예제 6: 포인터 연산

17

```
#include <stdio.h>
int main(void)
{
    int* ptr1 = (int*)0x0010;
    double* ptr2 = (double*)0x0010;

    printf("%p %p \n", ptr1 + 1, ptr1 + 2);
    printf("%p %p \n", ptr2 + 1, ptr2 + 2);

    printf("%p %p \n", ptr1, ptr2);
    ptr1++;
    ptr2++;
    printf("%p %p \n", ptr1, ptr2);
    return 0;
}
```

00000014	00000018
00000018	00000020
00000010	00000010
00000014	00000018



예제7: 포인터 연산

18

```
#include <stdio.h>
int main(void)
{
    int arr[3] = { 11, 22, 33 };
    int* ptr = arr;    // int * ptr=&arr[0]; 과 같은 문장

    printf("%d %d %d \n", arr[0], arr[1], arr[2]);
    printf("%d %d %d \n", ptr[0], ptr[1], ptr[2]);
    printf("%d %d %d \n", *arr, *(arr + 1), *(arr + 2));
    printf("%d %d %d \n", *ptr, *(ptr + 1), *(ptr + 2));

    return 0;
}
```

11	22	33
11	22	33
11	22	33
11	22	33

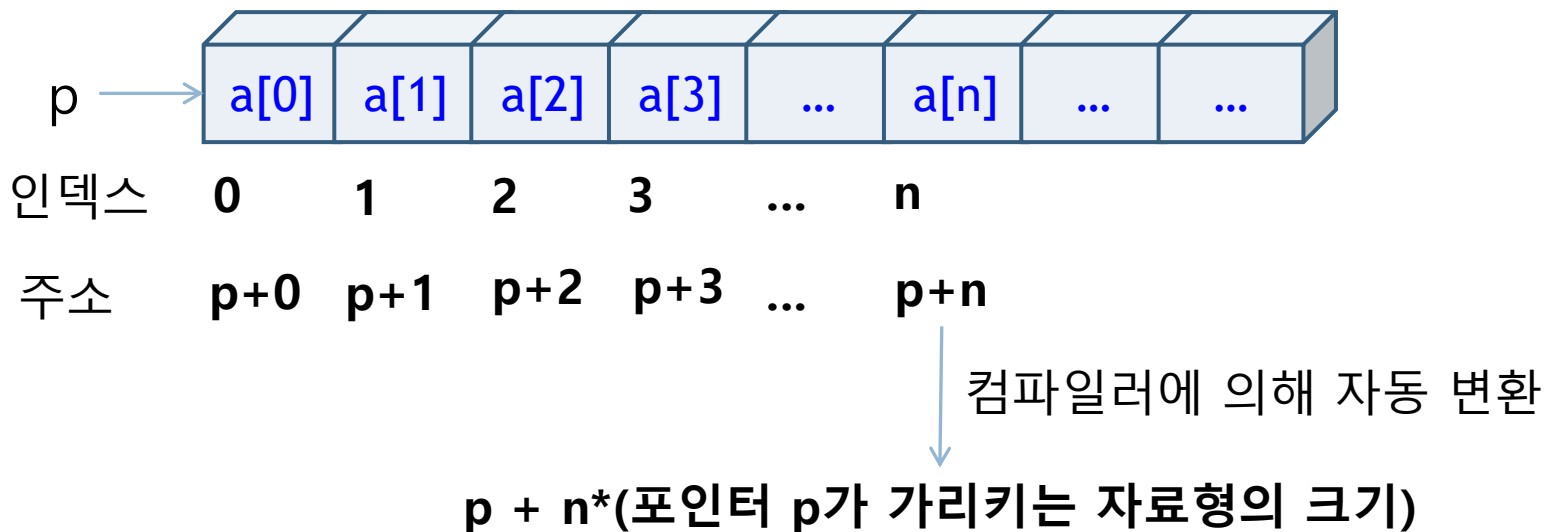
arr, ptr => 배열의 1번째 요소의 주소
arr+1, ptr+1 => 배열의 2번째 요소의 주소
arr+2, ptr+2 => 배열의 3번째 요소의 주소



포인터 연산의 의미

19

- 포인터에 더해지는 정수는 주소 p 에서 시작되는 배열의 **인덱스**로 해석하면 됨
- $p + n \rightarrow p$ 에서 시작하는 배열에서 $p[n]$ 의 주소



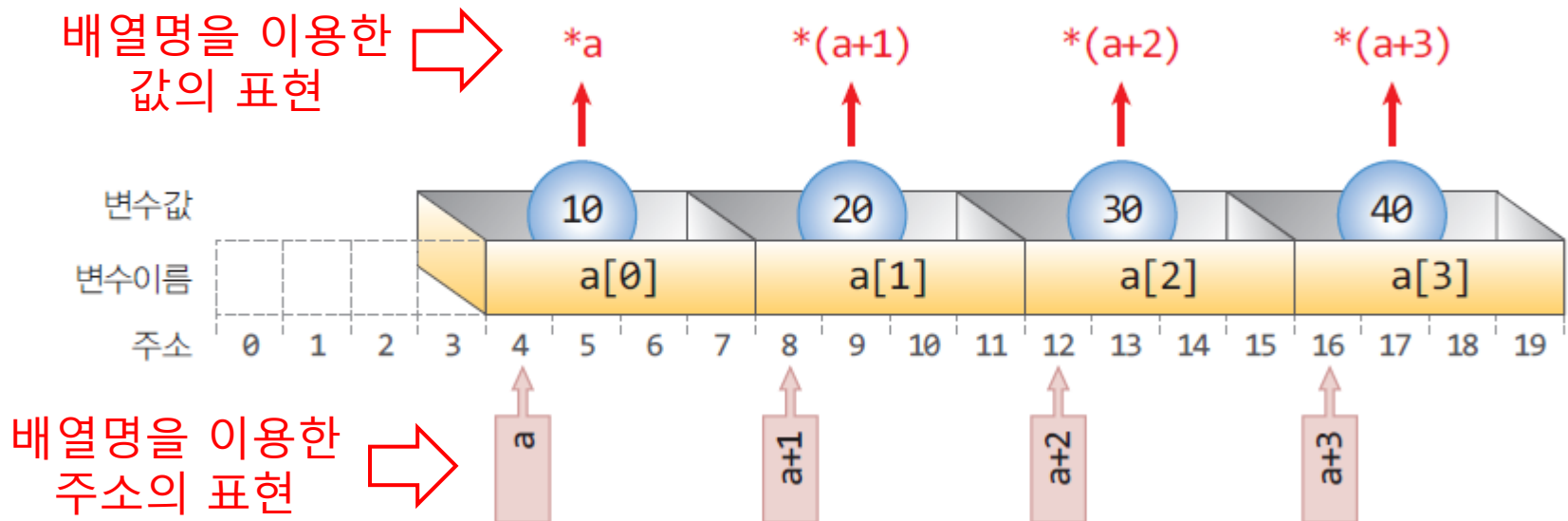


포인터와 배열

20

- 배열은 원소들이 메모리에서 연속된 공간에 할당됨
- 첫번째 요소의 주소만 알면 나머지 요소의 주소는 쉽게 계산 가능
- 배열 요소의 주소는 (배열명 + 인덱스) 로 표현 가능
- 배열 요소의 값은 $\ast(\text{배열명} + \text{인덱스})$ 로 표현 가능

```
int a[4] = {10,20,30,40};
```





예제 8: 포인터와 배열

21

```
#include <stdio.h>
int main(void)
{
    int a[ ] = { 10, 20, 30, 40, 50 };
    printf("a = %u\n", a);           // &a[0]
    printf("a + 1 = %u\n", a + 1);  // &a[1]
    printf("*a = %d\n", *a);        // a[0]
    printf("*(a+1) = %d\n", *(a+1)); // a[1]
    return 0;
}
```

```
a = 1245008
a + 1 = 1245012
*a = 10
*(a+1) = 20
```

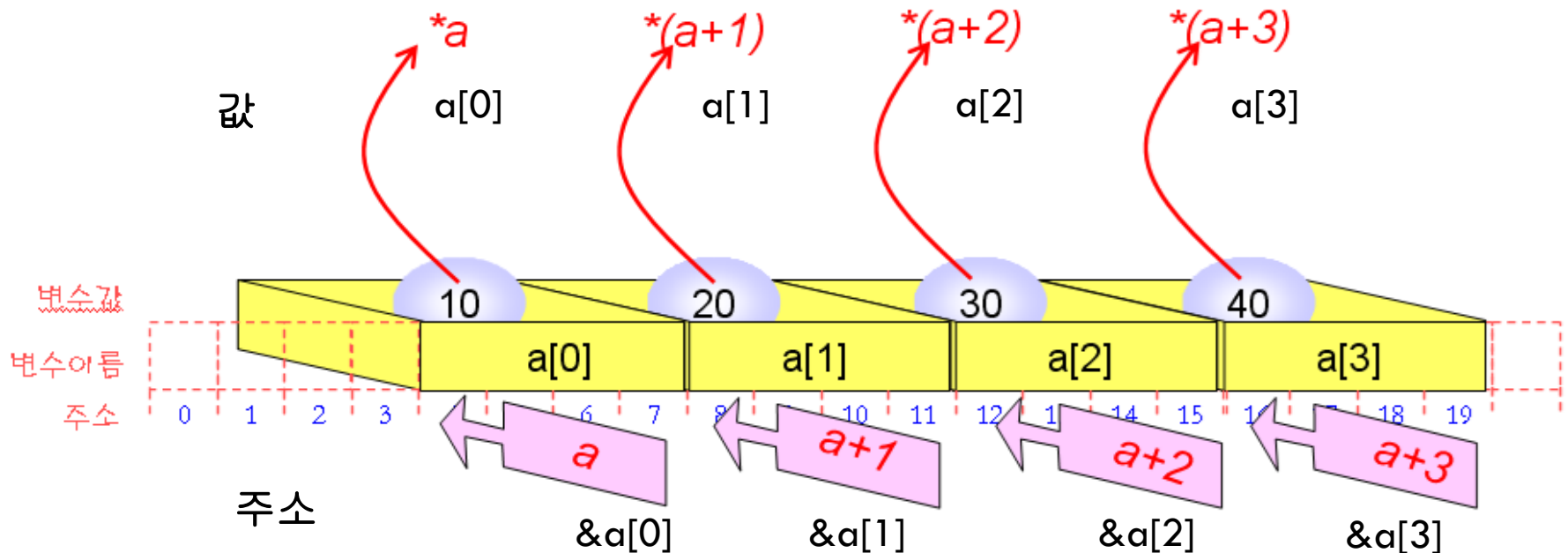


포인터와 배열

22

□ 포인터(주소)표현과 배열표현의 관계

	배열표현	포인터(주소) 표현
배열요소의 값	$a[i]$	$*(a+i)$
배열요소의 주소	$\&a[i]$	$a+i$





예제9: 포인터와 배열

23

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int a[] = { 10, 20, 30};
```

```
    printf("a[0]=%d a[1]=%d a[2]=%d \n", a[0], a[1], a[2]);
```

```
    printf("*a=%d *(a+1)=%d *(a+2)=%d \n", *a, *(a+1), *(a+2));
```

```
    printf("&a[0]=%d &a[1]=%d &a[2]=%d \n", &a[0], &a[1], &a[2]);
```

```
    printf("a=%d a+1=%d a+2=%d \n", a, a+1, a+2);
```

```
    return 0;
```

```
}
```

```
a[0]=10, a[1] =20, a[2]=30
```

```
*a = 10, *(a+1)=20, *(a+2)=30
```

```
&a[0] = 0788, &a[1] = 0792, &a[2] = 0796
```

```
a = 0788, a+1 = 0792, a+2 = 0796
```

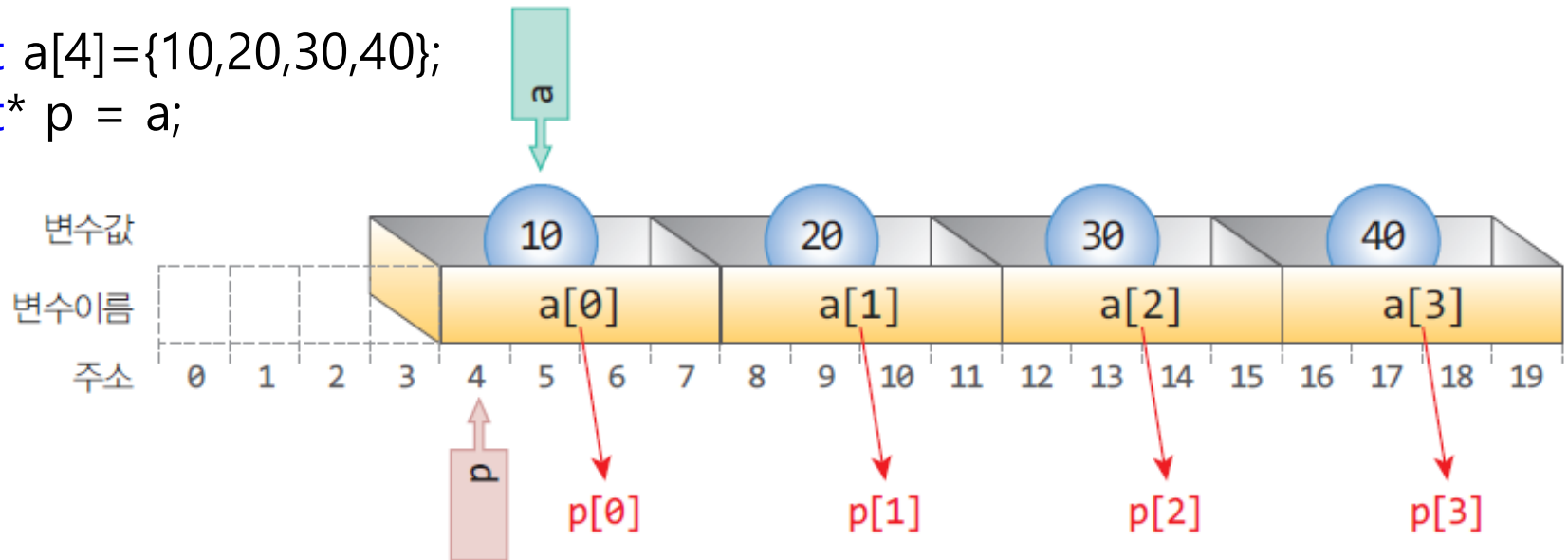


포인터를 배열처럼 사용

24

- 포인터도 배열명처럼 간주될 수 있고 배열과 똑같이 사용가능

```
int a[4]={10,20,30,40};  
int* p = a;
```



포인터를 이용한 각 요소의 주소?
포인터를 이용한 각 요소의 값?



예제 10: 포인터를 배열처럼 사용

25

```
#include <stdio.h>
int main(void)
{
    int a[] = { 10, 20, 30};
    int* p = a;
    printf("a[0]=%d a[1]=%d a[2]=%d \n", a[0], a[1], a[2]);
    printf("p[0]=%d p[1]=%d p[2]=%d \n", p[0], p[1], p[2]);
    p[0] = 60; p[1] = 70; p[2] = 80;
    printf("a[0]=%d a[1]=%d a[2]=%d \n", *a, *(a+1), *(a+2));
    printf("p[0]=%d p[1]=%d p[2]=%d \n", *p, *(p+1), *(p+2));

    return 0;
}
```

```
a[0]=10 a[1]=20 a[2]=30
p[0]=10 p[1]=20 p[2]=30
a[0]=60 a[1]=70 a[2]=80
p[0]=60 p[1]=70 p[2]=80
```

// 포인터를 배열명처럼 사용 가능

*p[0] => *(p+0) => *(a+0) => a[0]*

*p[1] => *(p+1) => *(a+1) => a[1]*

*p[2] => *(p+2) => *(a+2) => a[2]*



예제 1 1

26

```
#include <stdio.h>
int main(void)
{
    int a[] = { 10, 20, 30 }, i, sum = 0;
    int* p = a;
```

```
    for (i = 0; i < 3; i++)
        sum += a[i];
    printf("sum:%d\n", sum);
```

//배열표현

```
    sum = 0;
    for (i = 0; i < 3; i++)
        sum += *(a+i);
    printf("sum:%d\n", sum);
```

//배열명을 이용한 주소표현

```
sum:60
sum:60
sum:60
sum:60
```



예제 1 1

27

```
sum = 0;
for (i = 0; i < 3; i++)
    sum += p[i];    //포인터변수를 이용한 배열표현
printf("sum:%d\n", sum);
```

```
sum = 0;
for (i = 0; i < 3; i++)
    sum += *(p+i); //포인터 변수를 이용한 주소표현
printf("sum:%d\n", sum);
```

```
sum = 0;
for (i = 0; i < 3; i++)
    sum += *p++;    //증감연산자 이용한 주소표현
printf("sum:%d\n", sum);
```

```
return 0;
```

```
}
```

```
sum:60
sum:60
sum:60
sum:60
sum:60
```



실습과제 1

28

- 배열명의 의미를 설명하라.
- 배열표현과 포인터(주소) 표현의 변환 공식을 설명하라.
- 배열의 첫번째 요소의 주소만 알면 모든 원소의 주소를 계산할 수 있다. 방법을 설명하십시오.
- 배열의 첫번째 요소의 주소만 알면 모든 원소의 값을 계산할 수 있다. 방법을 설명하십시오.
- 포인터에 대해 가능한 연산의 종류와 의미를 설명하라.
- 포인터 연산을 기존의 산술연산과 다르게 정의한 이유는 무엇일까?

- 다음 코드의 배열 표현을 포인터(주소) 표현으로 변경하시오.

```
#include <stdio.h>
int main(void)
{
    int grade[5];
    int sum = 0, i, average;
    for(i = 0; i < 5; i++)
    {
        printf("성적을 입력하시오: ");
        scanf("%d", &grade[i]);
    }
    for(i = 0; i < 5; i++)
        sum += grade[i];

    average = sum / 5.0;
    printf("성적 평균 = %d\n", average);

    return 0;
}
```

```
성적을 입력하시오: 10<엔터>
성적을 입력하시오: 20<엔터>
성적을 입력하시오: 30<엔터>
성적을 입력하시오: 40<엔터>
성적을 입력하시오: 50<엔터>
성적 평균 = 30
```



실습과제3

30

- 증감연산자 $a++$, $++a$ 의 차이를 설명하시오.
- 다음 코드의 문제점을 설명하고 포인터를 이용하여 문제를 해결하시오.

```
#include <stdio.h>
int main(void)
{
    int a[] = { 10, 20, 30 }, i, sum = 0;

    for (i = 0; i < 3; i++)
        sum += *a++;
    printf("sum:%d\n", sum);
    return 0;
}
```

sum:60

- 교재 299페이지 문제1에서 **배열의 초기값을 키보드로부터 5개의 실수를** 입력 받아 저장하고 나머지는 문제와 동일하게 푸시오.
- 키보드로부터 입력 받아 저장할 때도 문제처럼 포인터 변수를 증가시키는 형태의 연산을 이용하여 작성하시오.-> 증감연산자 사용

- 교재 300페이지 문제2에서 **배열의 초기값을 키보드로부터 5개의 실수를** 입력 받아 저장하고 나머지는 문제와 동일하게 푸시오.
- 키보드로부터 입력 받아 저장할 때도 문제처럼 포인터 변수를 덧셈 연산을 하는 형태의 연산을 이용하여 작성하시오.
- 예제 11처럼 **5가지 방식으로** 코드를 모두 작성해볼 것



과제제출방법

33

- 아래 주소의 작성방법대로 github에 제출할 것
 - ▣ <https://github.com/2sungryul/cprogramming/tree/main>
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```